Nama : Akhmad Hasan Kurniawan
NIM : 244107060115
Kelas : SIB-2A

JOBSHEET W07

INHERITANCE, OVERLOADING, OVERRIDING

PERCOBAAN 1 (extends)

A. TAHAPAN PERCOBAAN

1. Buatlah sebuah parent class dengan nama Pegawai. Lalu buat constructor tanpa parameter.

```
public class Pegawai {

    public Pegawai() {
        System.out.println("Ini constructor class Pegawai");
    }
}
```

2. Buatlah subclass dari class Pegawai dengan nama Dosen, kemudian buat juga constructor tanpa parameter.

```
public class Dosen extends Pegawai {

    public Dosen() {
        System.out.println("Ini constructor class Dosen");
    }
}
```

3. Buatlah main class, misal InheritanceDemo.java, lakukan instansiasi objek baru bernama dosen1 dari class Dosen.

```
public static void main(String[] args) {
    Dosen dosen1 = new Dosen();
}
```

4. Run programnya kemudian amati hasilnya.

```
Ini constructor class Pegawai
Ini constructor class Dosen
```

B. PERTANYAAN

1. Pada percobaan 1 diatas, tentukan child class dan parent class!
   - Parent : Pegawai
   - Clid : Dosen
2. Kata kunci apa yang membuat child class dan parent class tersebut memiliki relasi inheritance?
   extends
3. Tambahkan keyword final pada class Pegawai. Run program kemudian amati hasilnya. Mengapa terjadi error?
   Terjadi error karena class yang diberi keyword **final** tidak dapat diwariskan (inherit) oleh class lain. Akibatnya class Dosen tidak bisa menggunakan extends Pegawai.

4. Hapus kembali keyword final.
5. Berdasarkan output yang ditampilkan oleh program, ada berapa constructor yang dipanggil saat instansiasi objek dosen1 dilakukan? Constructor class mana yang lebih dulu dieksekusi?
   - Ada konstruktor yg dipanggil
   - Construktor yang dieksekusi terlebih dahulu adalah constructor class Pegawai (parent class).
   - Setelah itu constructor class Dosen (clild class) dieksekusi.

PERCOBAAN 2 (Pewarisan)

A. TAHAPAN PERCOBAAN

1. Tambahkan atribut nip, nama, dan gaji serta method getInfo() pada class Pegawai

```
public class Pegawai {
    public String nip;
    public String nama;
    public double gaji;

    public Pegawai() {
        System.out.println("Ini constructor class Pegawai");
    }

    public String getInfo() {
        String info = "";
        info += "NIP    : " + nip + "\n";
        info += "Nama   : " + nama + "\n";
        info += "Gaji   : " + gaji + "\n";

        return info;
    }
}
```

2. Tambahkan pula atribut NIDN pada class Dosen

```
public class Dosen extends Pegawai {
    public String nidn;

    public Dosen() {
        System.out.println("Ini constructor class Dosen");
    }
}
```

3. Pada class InheritanceDemo.java tuliskan baris kode berikut:

```
public static void main(String[] args) {
    Dosen dosen1 = new Dosen();

    dosen1.nama = "Yansy Ayuningtyas";
    dosen1.nip = "34329837";
    dosen1.gaji = 3000000;
    dosen1.nidn = "1989432439";

    System.out.println(dosen1.getInfo());
}
```

4. Run program kemudian amati hasilnya

```
Ini constructor class Pegawai
Ini constructor class Dosen
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
```

B. PERTANYAAN

1. Pada percobaan 2 diatas, apakah program dapat berhasil dijalankan ataukah terjadi error?
   Berhasil dijalankan
2. Jika program berhasil dijalankan, mengapa tidak terjadi error pada assignment/pengisian nilai atribut tersebut pada kelas dosen?
   Karena pada kelas dosen telah ditambajkan atribut NIDN
3. Jika program berhasil dijalankan, mengapa tidak terjadi error pada pemanggilan method getInfo() oleh object dosen1 padahal tidak ada deklarasi method getInfo() pada class Dosen?
   Method getInfo() dapat dipanggil oleh objek dosen1 karena method tersebut **diwariskan dari class Pegawai ke class Dosen melalui mekanisme inheritance (extends).**

PERCOBAAN 4 (keyword super)

A. TAHAPAN PERCOBAAN

1. Butlah method getAllInfo() pada class Dosen

```
public String getAllInfo(){
    String info = "";
    info += "NIP         : " + nip + "\n";
    info += "Nama        : " + nama + "\n";
    info += "Gaji        : " + gaji + "\n";
    info += "NIDN        : " + nidn + "\n";

    return  info;
}
```

2. Lakukan pemanggil method getAllInfo() oleh object dosen1 pada main function.

```
public static void main(String[] args) {
    Dosen dosen1 = new Dosen();

    dosen1.nama = "Yansy Ayuningtyas";
    dosen1.nip = "34329837";
    dosen1.gaji = 3000000;
    dosen1.nidn = "1989432439";

    System.out.println(dosen1.getAllInfo());
}
```

3. Run program kemudian amati hasilnya

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

4. Lakukan modifikasi method getAllInfo() pada class Dosen

```
public String getAllInfo(){
    String info = "";
    info += "NIP         : " + this.nip + "\n";
    info += "Nama        : " + this.nama + "\n";
    info += "Gaji        : " + this.gaji + "\n";
    info += "NIDN        : " + this.nidn + "\n";

    return  info;
}
```

5. Run program kemudian bandingkan hasilnya dengan langkah no 2.

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

6. Lakukan modifikasi method getAllInfo() pada class Dosen kembali

```
public String getAllInfo(){
    String info = "";
    info += "NIP         : " + super.nip + "\n";
    info += "Nama        : " + super.nama + "\n";
    info += "Gaji        : " + super.gaji + "\n";
    info += "NIDN        : " + super.nidn + "\n";

    return  info;
}
```

7. Run program kemudian bandingkan hasilnya dengan progam pada no 1 dan no 4.

```
nidn cannot be resolved or is not a field
```

1. Lakukan modifikasi method getAllInfo() pada class Dosen kembali

```
public String getAllInfo(){
    String info = "";
    info += "NIP         : " + super.nip + "\n";
    info += "Nama        : " + super.nama + "\n";
    info += "Gaji        : " + super.gaji + "\n";
    info += "NIDN        : " + this.nidn + "\n";

    return  info;
}
```

2. Run program kemudian bandingkan hasilnya dengan progam pada no 2 dan no 4.

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

B. PERTANYAAN

1. Apakah terdapat perbedaan output pada langkah 1, 4, dan 8? Mengapa?
   Tidak ada, semua atiribut mengacu pada nilai atribut yang sama sehingga outputny tetap sama
2. Mengapa error terjadi pada program no 6?
   karena kode super.nidn mencoba akses atribut nidn dari padahal atribut nidn hanya terdapat pada class dosen sehingga compiler menampilkan error karena pegawai tidak memiliki atribut nidn
3. Ubah atribut nip pada class Pegawai menjadi private, kemudian coba run program, mengapa terjadi compile error? Berdasarkan percobaan tersebut, bagaimana cara menentukan atribut dan method yang akan diwariskan oleh parent class ke child class?
   Terjadi compile error karena atribut private hanya dapat diakses di dalam class yang

mendeklarasikannya.

4. Kembalikan access level modifier nip menjadi public

PERCOBAAN 5 (super & overriding)

C. TAHAPAN PERCOBAAN

1. Lakukan modifikasi berikut pada method getAllInfo(). Run program kemudian amati hasilnya.

```
public String getAllInfo(){
    String info = getInfo();
    info += "NIDN       : " + nidn;

    return info;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

2. Lakukan modifikasi berikut pada method getAllInfo(). Run program kemudian amati hasilnya

```
public String getAllInfo(){
    String info = this.getInfo();
    info += "NIDN       : " + nidn;

    return info;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

3. Lakukan modifikasi berikut pada method getAllInfo(). Run program kemudian amati hasilnya

```
public String getAllInfo(){
    String info = super.getInfo();
    info += "NIDN       : " + nidn;

    return info;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

4. Tambahkan method getInfo() pada class Dosen dan modifikasi method getAllInfo() sebagai berikut

```
public class Dosen extends Pegawai {
    public String nidn;

    public Dosen() {
        System.out.println("Objek dari class Dosen dibuat");
    }

    public String getInfo(){
        return "NIDN        : " + this.nidn + "\n";
    }

    public String getAllInfo(){
        String info = super.getInfo();
        info += this.getInfo();

        return info;
    }
}
```

D. PERTANYAAN

1. Apakah ada perbedaan method getInfo() mana yang dipanggil pada langkah 1, 2, dan 3?
   iya ada
2. Pada potongan kode berikut di langkah 4, apakah super.getInfo() dan this.getInfo() memanggil method yang sama? Jelaskan!

```
String info = this.getInfo();
info += super.getInfo();
```

   Tidak selalu sama, this.getInfo() itu memanggil kls dosen sedangkan super.getInfo() memanggil kls pegawai

3. Pada method manakah terjadi overriding? Jelaskan!
   terjadi ketika method public String getInfo(), karena method tersebut memiliki nama method yg sama, tipe return sama dan parameter sama(tidak ada parameter)
4. Tambahkan keyword final pada method getInfo() di class Pegawai. Apakah program dapat dicompile? Mengapa?
   tidak dapat di compile karena method yang diberi keyword final tidak boleh dioveride oleh child class
5. Hapus kembali keyword final yang baru ditambahkan.

PERCOBAAN 6 (overloading)

A. TAHAPAN PERCOBAAN

1. Tambahkan constructor baru untuk class Dosen sebagai berikut

```
public Dosen(String nip, String nama, double gaji, String nidn) {
    System.out.println("Ini constructor class Dosen yang berparameter");
}
```

2. Modifikasi class InheritanceDemo untuk menginstansiasi object baru dengan nama dosen2 dengan constructor yang berparameter. Run program kemudian amati hasilnya.

```
public static void main(String[] args) {
    Dosen dosen2 = new Dosen("34329837","Yansy Ayuningtyas",3000000,"1989432439");
    System.out.println(dosen2.getAllInfo());
}
```

```
Ini constructor class Dosen yang berparameter
nip : null
nama : null
gaji : 0.0
NIDN  : null
```

B. PERTANYAAN

1. Bagaimana hasil nilai nip, nama, gaji, dan nidn yang ditampilkan pada langkah 2? Mengapa demikian?
   Semuanya kosong: null untuk tulisan dan 0.0 untuk gaji. Alasannya, constructor yang berparameter itu hanya **mencetak tulisan**, tidak memasukkan data ke dalam variabelnya. Bayangkan kamu dikasih 4 kotak, tapi kamu cuma bilang "terima kasih" dan tidak memasukkan isi apa pun ke kotaknya. Kotaknya tetap kosong.
2. Jelaskan apakah 2 constructor pada class Dosen memiliki parameter yang sama?
   Constructor pertama Dosen() tidak punya parameter. Constructor kedua punya 4 parameter: String nip, String nama, double gaji, String nidn.
3. Pada method apa terjadi overloading? Jelaskan
   Pada **constructor Dosen**. Namanya sama-sama Dosen, tapi parameternya beda (yang satu kosong, yang satu ada 4). Ini seperti dua tombol bernama sama tapi cara pakainya beda.
4. Kesimpulannya, apakah method overloading dapat dilakukan terhadap constror?
   Bisa. Satu class boleh punya banyak constructor, asalkan parameternya berbeda (jumlah, tipe, atau urutanny).

PERCOBAAN 7 (super - constructor)

C. TAHAPAN PERCOBAAN

1. Modifikasi constructor berparameter pada class Dosen sebagai berikut. Run program kemudian amati hasilnya.

```
public Dosen(String nip, String nama, double gaji, String nidn){
    this.nip = nip;
    this.nama = nama;
    this.gaji = gaji;
    this.nidn = nidn;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

2. Modifikasi constructor pada class Dosen sebagai berikut. Run program kemudian amati hasilnya.

```
public Dosen(String nip, String nama, double gaji, String nidn){
    super.nip = nip;
    super.nama = nama;
    super.gaji = gaji;
    this.nidn = nidn;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

3. Modifikasi constructor pada class Dosen sebagai berikut. Run program kemudian amati hasilnya.

```
public Dosen(String nip, String nama, double gaji, String nidn){
    super();
    super.nip = nip;
    super.nama = nama;
    super.gaji = gaji;
    this.nidn = nidn;
}
```

```
nip : 34329837
nama : Yansy Ayuningtyas
gaji : 3000000.0
nidn : 1989432439
```

D. PERTANYAAN

1. Apakah terdapat perbedaan hasil pada langkah 1 dan 2? Mengapa?
   Tidak ada. Dosen tidak punya variabel nip, nama, dan gaji sendiri. Semuanya diwarisi dari Pegawai, jadi this.nip dan super.nip menunjuk ke kotak yang sama.
2. Apa yang dilakukan oleh baris program `super();` pada langkah 3?
   Baris itu **memanggil constructor milik class Pegawai** (induknya) yang tanpa parameter. Jadi sebelum Dosen mengerjakan tugasnya, ia "menyapa orang tuanya" dulu.
3. Apakah terdapat pemanggilan constructor superclass pada langkah 1, 2, dan 3?
   Ada di ketiganya. Di langkah 1 dan 2, Java **diam-diam menambahkan super();** di baris pertama. Itu sebabnya tulisan "Ini constructor class Pegawai" tetap muncul. Di langkah 3, super(); ditulis **terang-terangan** oleh kita.

4. Hapus/comment constructor tanpa parameter dari class Pegawai. Tambahkan constructor baru untuk class Pegawai sebagai berikut.

```
public class Pegawai {
    public String nip;
    public String nama;
    public double gaji;

    // public Pegawai() {
    //     System.out.println("Ini constructor class Pegawai");
    // }

    public Pegawai(String nip, String nama, double gaji) {
        this.nip = nip;
        this.nama = nama;
        this.gaji = gaji;
    }

    public String getInfo() {
        String info = "";
        info += "NIP    : " + nip + "\n";
        info += "Nama   : " + nama + "\n";
        info += "Gaji   : " + gaji + "\n";

        return info;
    }
}
```

Coba run program kemudian amati hasilnya. Mengapa terjadi compile error?

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        Implicit super constructor Pegawai() is undefined. Must explicitly invoke another constructor
        Implicit super constructor Pegawai() is undefined. Must explicitly invoke another constructor
```

5. Modifikasi constructor pada class Dosen sebagai berikut.

```
public Dosen(String nip, String nama, double gaji, String nidn){
    super(nip, nama, gaji);
    this.nidn = nidn;
}
```

Run program kemudian amati hasilnya. Mengapa tidak terjadi error? Apakah super(nip, nama, gaji) pada langkah ini dan super() pada langkah sebelumnya memanggil method yang sama?

```
NIP    : 34329837
Nama   : Yansy Ayuningtyas
Gaji   : 3000000.0
NIDN   : 1989432439
```

- Tidak terjadi error karena constructor Dosen kini memanggil super(nip, nama, gaji), yang sesuai dengan constructor berparameter (String nip, String nama, double gaji) pada class Pegawai. Constructor yang dipanggil tersedia, sehingga proses compile berhasil.
- Tidak sama. super() memanggil constructor Pegawai tanpa parameter, sedangkan super(nip, nama, gaji) memanggil constructor Pegawai yang memiliki tiga parameter.

Keduanya merupakan constructor yang berbeda karena daftar parameternya berbeda (overloading).

6. Modifikasi constructor pada class Dosen sebagai berikut.

```
public Dosen(String nip, String nama, double gaji, String nidn){
    this.nidn = nidn;
    super(nip, nama, gaji);
}
```

Run program kemudian amati hasilnya. Mengapa terjadi error?
karena memanggil super harus jadi baris pertama dulu di contuctor

7. Kembalikan posisi pemanggilan constructor superclass pada baris pertama.

TUGAS

1. Gunakan studi kasus yang telah Anda buat pada materi Inheritance
2. Lakukan method overloading dengan membuat 2 constructor yaitu constructor tanpa parameter dan constructor berparameter, baik pada subclass maupun superclass, baik pada subclass maupun superclass
3. Panggil c constructor super() berparameter untuk membuat object dari superclass dari constructor subclass.
4. Lakukan method overriding dengan membuat method dengan nama dan parameter yang sama pada superclass dan subclass.
5. Lakukan instansiasi 2 objek subclass pada main() function kemudian print info nya.

Class Kopi

```
package Tugas;

public class Kopi {

    public String nama;
    public double harga;

    // Constructor tanpa parameter
    public Kopi() {
        System.out.println("Constructor Kopi (tanpa parameter) dipanggil");
        this.nama = "Kopi Biasa";
        this.harga = 0;
    }

    // Constructor berparameter
    public Kopi(String nama, double harga) {
        System.out.println("Constructor Kopi (berparameter) dipanggil");
        this.nama = nama;
        this.harga = harga;
    }
```

```
    // Method yang nanti di-override oleh subclass
    public String getInfo() {
        String info = "";
        info += "Nama Kopi : " + this.nama + "\n";
        info += "Harga     : Rp" + this.harga + "\n";
        return info;
    }
}
```

Class Kopilatte

```
package Tugas;

public class KopiLatte extends Kopi {

    public String jenisSusu;

    // Constructor tanpa parameter
    public KopiLatte() {
        super();
        System.out.println("Constructor KopiLatte (tanpa parameter)
dipanggil");
        this.jenisSusu = "Susu Sapi";
    }

    // Constructor berparameter
    public KopiLatte(String nama, double harga, String jenisSusu) {
        super(nama, harga); // memanggil constructor Kopi yang berparameter
        System.out.println("Constructor KopiLatte (berparameter) dipanggil");
        this.jenisSusu = jenisSusu;
    }

    // Overriding: nama dan parameter sama dengan getInfo() di Kopi
    @Override
    public String getInfo() {
        String info = super.getInfo();
        info += "Jenis Susu: " + this.jenisSusu + "\n";
        return info;
    }
}
```

Class Kopiespresso

```
package Tugas;

public class KopiEspresso extends Kopi {

    public int jumlahShot;
```

```
    // Constructor tanpa parameter
    public KopiEspresso() {
        super();
        System.out.println("Constructor KopiEspresso (tanpa parameter)
dipanggil");
        this.jumlahShot = 1;
    }

    // Constructor berparameter
    public KopiEspresso(String nama, double harga, int jumlahShot) {
        super(nama, harga); // memanggil constructor Kopi yang berparameter
        System.out.println("Constructor KopiEspresso (berparameter)
dipanggil");
        this.jumlahShot = jumlahShot;
    }

    // Overriding: nama dan parameter sama dengan getInfo() di Kopi
    @Override
    public String getInfo() {
        String info = super.getInfo();
        info += "Jumlah Shot: " + this.jumlahShot + "\n";
        return info;
    }
}
```

Class Kopidemo

```
package Tugas;

public class KopiDemo {
    public static void main(String[] args) {
        KopiLatte latte = new KopiLatte("Kopi Latte", 25000, "Susu Oat");
        System.out.println(latte.getInfo());

        KopiEspresso espresso = new KopiEspresso("Kopi Espresso", 20000, 2);
        System.out.println(espresso.getInfo());
    }
}
```